Islamic University of Technology

Lab: Bipartite Matching Engineering

CSE 4810: Algorithm Engineering

Group 3 — Bipartite Maximum Matching

ASH, ABT(PT)

August 6, 2026

General Instruction

This lab consists of **one focused task** and is designed to fit a **1.5-hour** session. Written derivations are replaced by **live implementation, profiling, and verbal defense**. You must:

Step 1. Complete the `// TODO` sections in the provided C++ source file — this includes both **building the super-source/super-sink flow network** and **implementing Max-Flow from scratch**.

Step 2. Compile and run the program, feeding it the exact sanity-check and benchmark inputs specified.

Step 3. Produce the requested live artifact — a matching size, a matched-pairs listing, and a runtime plot — and keep your terminal/IDE open.

Step 4. Be ready to defend your engineering choices verbally using the artifact on screen; no written report is collected.

Implementation (Max-Flow Reduction)	∼50 minutes
Sanity Check	∼15 minutes
Live Profiling	∼25 minutes

Protect implementation time first. A correct matching on the sanity check matters far more than a fully-annotated plot.

Files Provided

`BipartiteMatching.cpp`	Max-Flow-based Matching skeleton
`plotter_group3.py`	Plots runtime data

Task: Bipartite Maximum Matching via Max-Flow (Super Source / Super Sink)

Objective: Reduce Bipartite Maximum Matching to a Max-Flow problem by attaching a super source to every left vertex and a super sink to every right vertex, then solving with a from-scratch Edmonds-Karp implementation.

Key Idea — Matching as Max-Flow

Build a flow network with a super **source** s connected to every left vertex, and a super **sink** t connected to every right vertex, keeping the original bipartite edges (left → right) in between. Running Max-Flow from s to t gives a flow value that equals the size of the maximum matching, and the saturated left→right edges reveal the matching itself — *no dedicated matching algorithm is needed at all*, only the Max-Flow engine you already know how to build.

It is tempting to give the source→left and right→sink edges *infinite* capacity, since intuitively they "shouldn't be the bottleneck." This is **backwards**. The capacity-1 constraint at exactly these boundary edges is what forces each left vertex to send at most 1 unit of flow (matched at most once) and each right vertex to receive at most 1 unit (matched at most once). The *original* left→right edges are the ones that can safely be given a large/infinite capacity, since the boundary edges already cap the flow through each vertex at 1 — giving the boundary edges infinite capacity instead would let a single left vertex be "matched" to several right vertices simultaneously, which is not a valid matching.

source → left(i)	capacity **1**
right(j) → sink	capacity **1**
left(i) → right(j)	capacity `INF_CAP` (large / effectively unlimited)

0.1 TODO 1: Implement `bfs_augmenting_path(source, sink, parentEdge)`

[Hint] *Standard BFS, traversable only along edges with* `capacity - flow > 0`*. Record* `parentEdge[v] = {u, edge_index}` *for every visited vertex.*

0.2 TODO 2: Implement `max_flow(source, sink)`

[Hint] *Loop: call* `bfs_augmenting_path`*. If it fails, stop. Otherwise find the bottleneck along the path (minimum residual capacity), push that much flow onto every edge on the path, and subtract it from every reverse edge. Accumulate the bottleneck into the total.*

0.3 TODO 3: Implement `extract_matching(nLeft, nRight)`

[Hint] *Call this only after* `max_flow` *has fully terminated. Scan every left→right edge (vertex* `i+1` *to vertex* `nLeft+1+j`*) and collect* `(i, j)` *whenever* `flow > 0` *— each such edge carries exactly 1 unit, never more, because of the capacity-1 boundary edges.*

0.4 TODO 4: Implement `build_matching_network(nLeft, nRight, adj)`

[Hint] *Vertex 0 is the source, left* i = *vertex* $i+1$*, right* j = *vertex* $nLeft+1+j$*, vertex* $nLeft+nRight+1$ *is the sink. Add the three edge types exactly as specified in the capacity box above.*

Sanity Check — Expected Output

```
--- BIPARTITE MATCHING (MAX-FLOW) SANITY CHECK ---
Max-Flow value (= matching size) : 4 (Expected 4)
Matched pairs (left, right):
  left 0 <-> right 1
  left 1 <-> right 0
  left 2 <-> right 2
  left 3 <-> right 3
-------------------------------------------------
--- SECOND SANITY CHECK (deficient matching) ---
Computed matching size : 2 (Expected 2)
-------------------------------------------------
```

Note on Exact Pairing

Your exact matched pairs for the first sanity check may differ from the listing above (e.g. which right vertex ends up with left vertex 0) — multiple maximum matchings can exist for the same graph. What must match exactly is the **matching size** (4), and every reported pair must be a genuine edge in the input graph.

Compile & Run

```
g++ -O2 -std=c++17 -o matching BipartiteMatching.cpp
./matching
```

Live Profiling

Runs to Perform

- `run_benchmark()` (already wired into `main`) times your implementation on square bipartite graphs ($|L| = |R| = n$) across increasing n, at two densities (sparse ≈ 0.05, dense ≈ 0.3). Run it and inspect `data_matching.txt`.
- **Plot it:** `python3 plotter_group3.py data_matching.txt` produces `matching_runtime.png`.

Core Engineering Defense (Verbal Check)

- **Why Capacity 1 at the Boundary?** Using the "Capacity Assignment" box above, explain in your own words what would go wrong (concretely, on your sanity-check graph) if `source -> left(i)` edges were given infinite capacity instead of capacity 1.
- **Why This Reduction Works:** Explain why the max-flow value between the super source and super sink is guaranteed to equal the size of the maximum matching — what correspondence exists between an augmenting path in this network and an augmenting alternating path in the original bipartite graph?
- **Cost of Generality:** This Max-Flow-based approach reuses the exact same engine as Group 1/2/4's tasks, at the cost of some runtime overhead compared to a matching-specific algorithm.

Point to your runtime plot and discuss: is that overhead worth it, and when might you prefer a dedicated matching algorithm instead?

The instructor may ask you to add a single edge connecting a currently-unmatched left vertex to a currently-unmatched right vertex in your sanity-check graph and predict, *before* re-running your code, how the max-flow value will change. Justify your answer using the augmenting-path logic of Max-Flow, not just re-running the program.